

FIT5215 Deep Learning

Advanced Sequential Models

Teaching team

Department of Data Science and AI
Faculty of Information Technology, Monash University
Email: **trunglm@monash.edu**



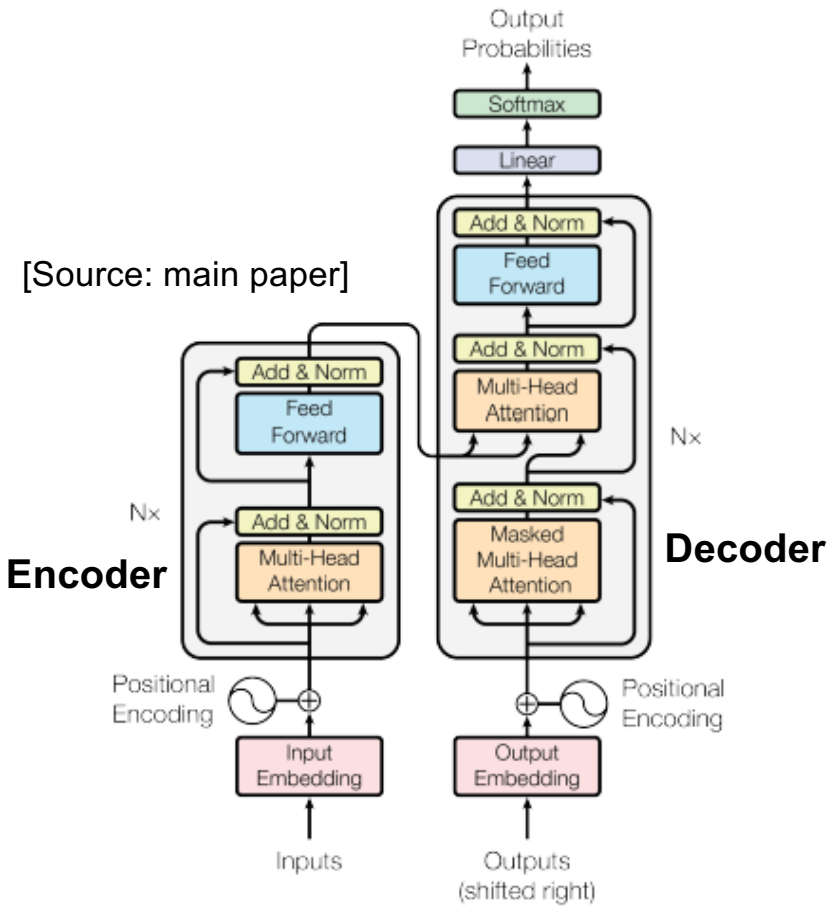
Outline

- Tutorial 09a: Transformers (*****)
- Tutorial 09b: Seq2seq for Machine Translation (***)

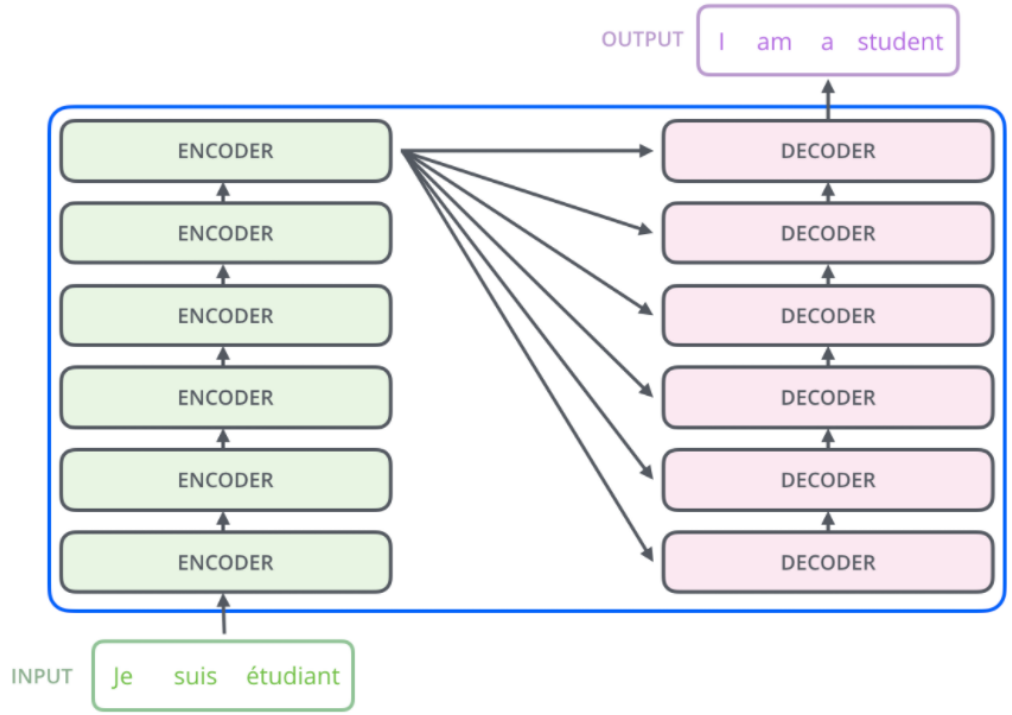
Transformers

Transformer – General Architecture

[Source: main paper]



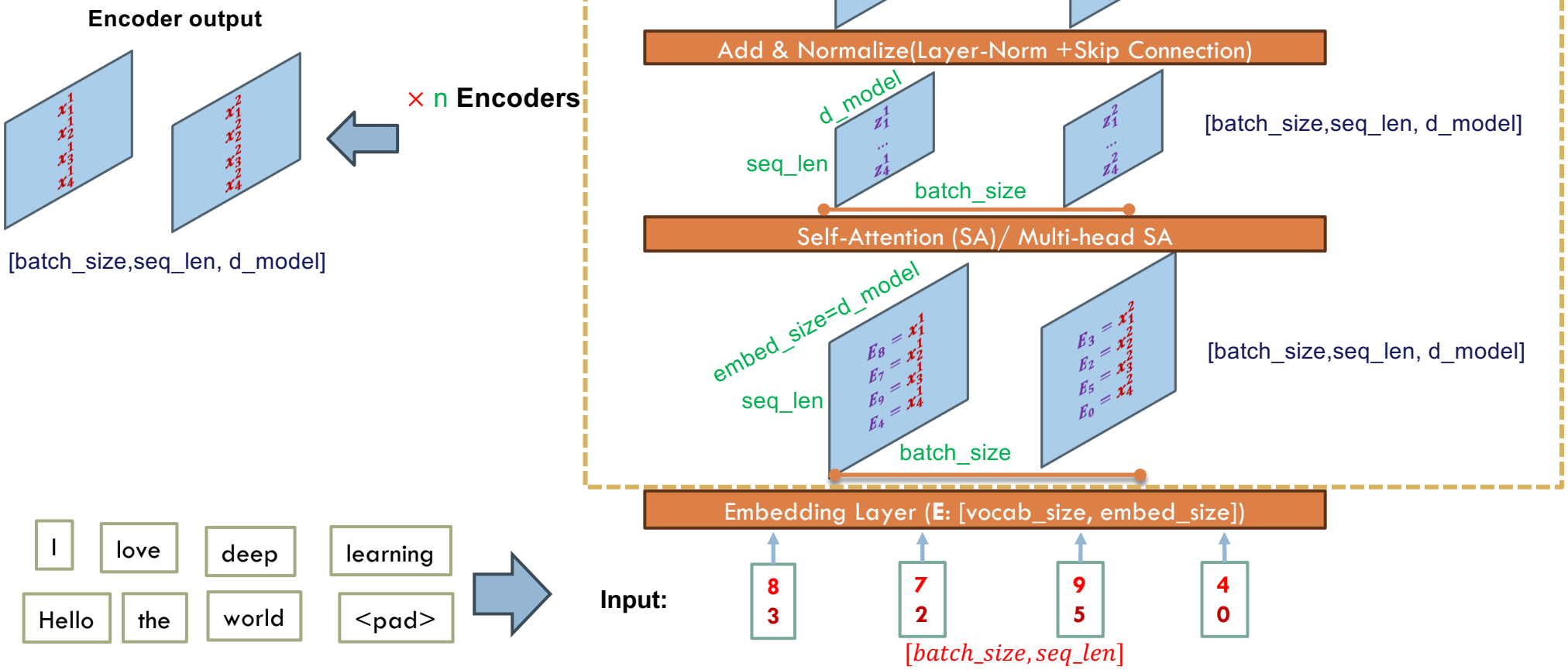
Model architecture



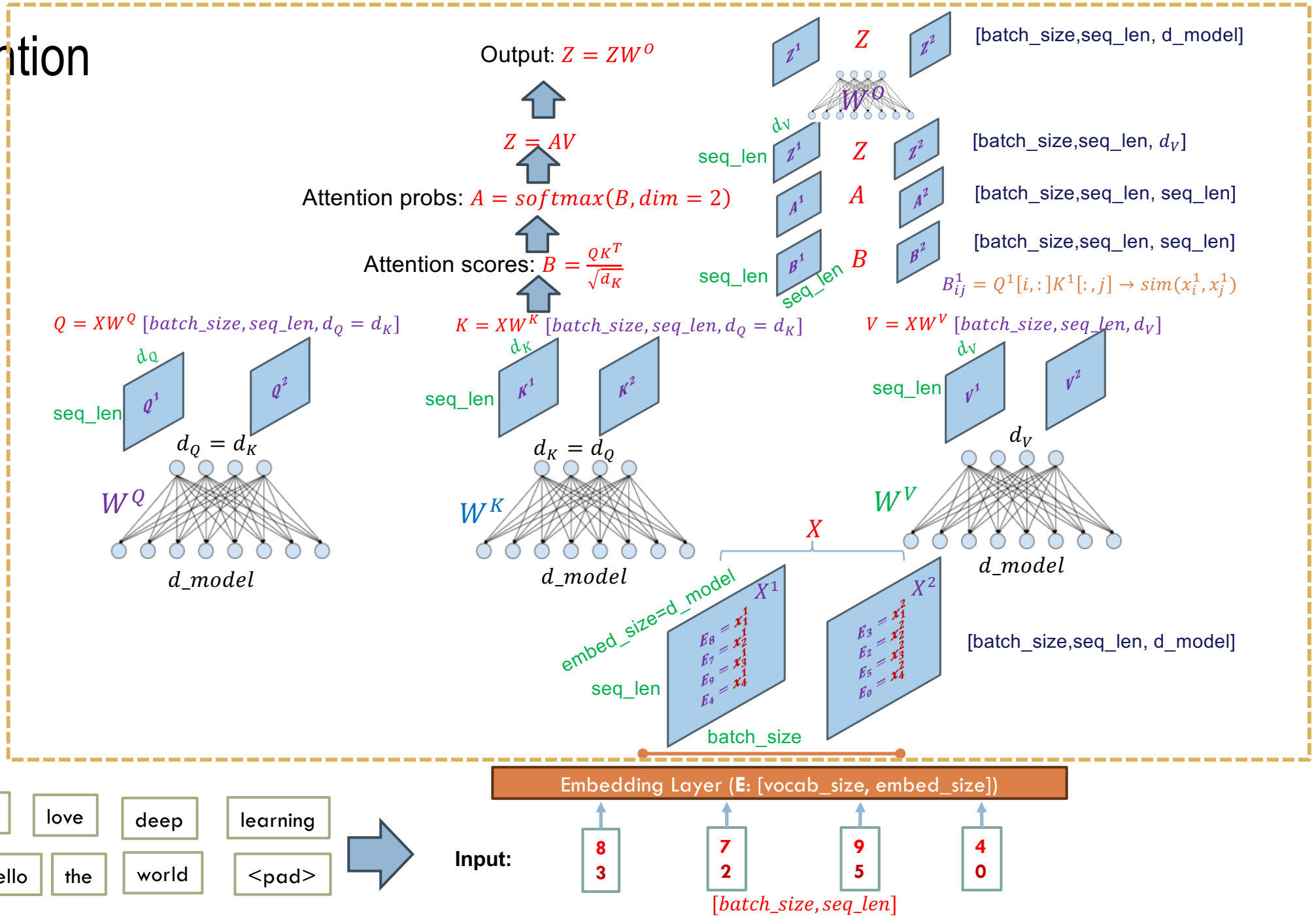
[Source: https://jalammar.github.io/illustrated-transformer/]

- Some **building blocks**
  - Positional encoding
  - Self-attention and Multi-Head Self-attention Attention
  - Masked Multi-Head Attention
  - Residual add and layer norm

Overview Architecture of Transformer Encoder

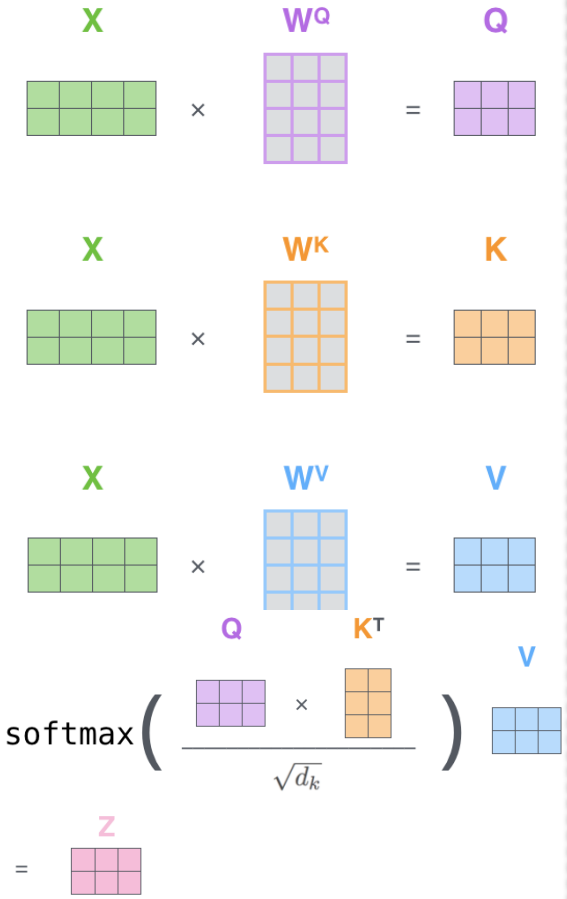


Self-Attention

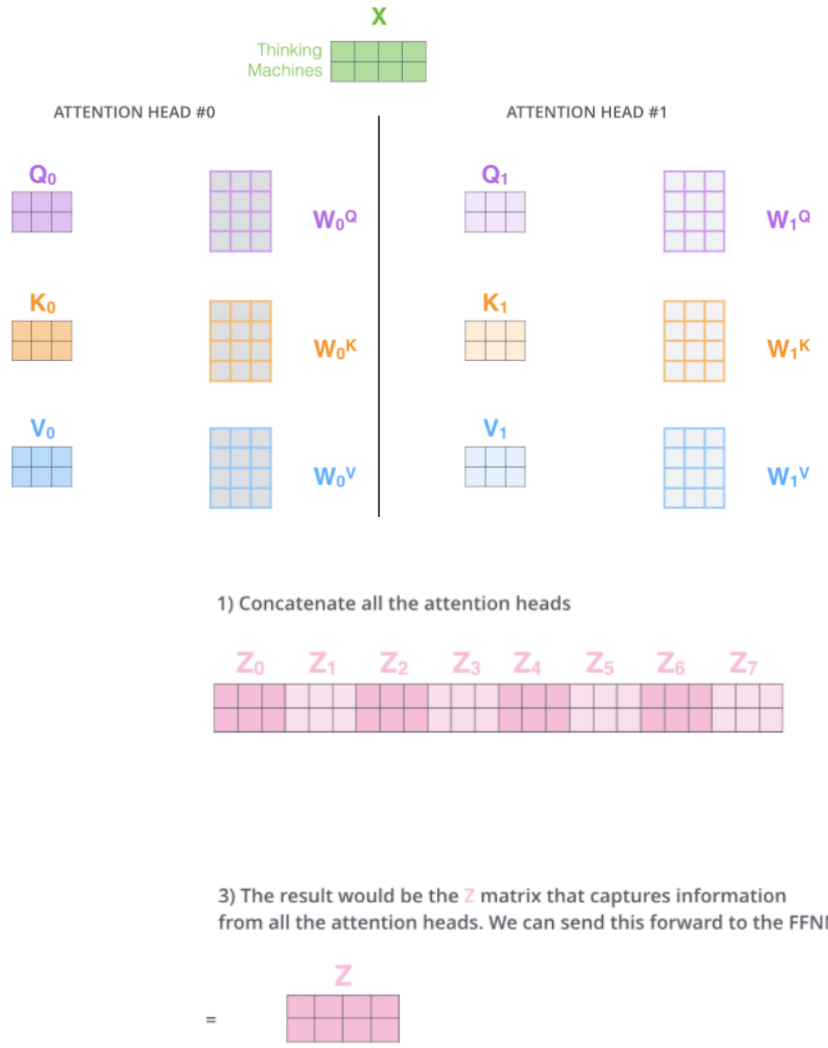


Transformer – Self Attention/Multi-Head Attention

Matrix calculation for One-Head



Matrix calculation for Multi-Head





[Source: https://jalammar.github.io/illustrated-transformer/]

Class `MultiHeadAttention`:

This class defines the multi-head attention mechanism using PyTorch's `nn.Module`. The `__init__` method initializes the necessary components, and the `forward` method defines how inputs are passed through the mechanism.

`__init__(self, d_model, num_heads)`:

This initializes the layer with key parameters.

1. **Parameters**:
   - `d_model`: The dimensionality of the model. For example, in BERT, this is typically 768 for base models.
   - `num_heads`: The number of attention heads, i.e., how many different sets of attention mechanisms the model uses.
2. **Dimension Handling**:
   - `self.d_k = d_model // num_heads`: Each head operates on a smaller subspace of the model's total dimension. This calculates the dimensionality (`d_k`) for each head by dividing the model's total dimension (`d_model`) by the number of heads.
3. **Linear Layers**:
   - `self.W_q`, `self.W_k`, `self.W_v`: These `nn.Linear` layers transform the input into queries (`Q`), keys (`K`), and values (`V`) for the attention mechanism.
   - `self.W_o`: After attention, the outputs from all heads are combined and passed through this linear transformation layer to match the original input dimension (`d_model`).

```python
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super(MultiHeadAttention, self).__init__()
        # Ensure that the model dimension (d_model) is divisible by the number of heads
        assert d_model % num_heads == 0, "d_model must be divisible by num_heads"
        # Initialize dimensions
        self.d_model = d_model # Model's dimension
        self.num_heads = num_heads # Number of attention heads
        self.d_k = d_model // num_heads # Dimension of each head's key, query, and value
        # Linear layers for transforming inputs
        self.W_q = nn.Linear(d_model, d_model) # Query transformation
        self.W_k = nn.Linear(d_model, d_model) # Key transformation
        self.W_v = nn.Linear(d_model, d_model) # Value transformation
        self.W_o = nn.Linear(d_model, d_model) # Output transformation
```

`scaled_dot_product_attention(self, Q, K, V, mask=None)`:

This function implements the core **Scaled Dot-Product Attention** mechanism.

1. **Attention Scores**:
   - `attn_scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)`: Computes the dot product between queries (`Q`) and keys (`K`) and scales the result by dividing by the square root of the dimensionality (`d_k`). This scaling helps stabilize gradients during training.
2. **Masking**:
   - `attn_scores = attn_scores.masked_fill(mask == 0, -1e9)`: If a mask is provided (e.g., to mask out padding tokens or prevent attention to future tokens), it replaces the masked positions with a very large negative value (`-1e9`), ensuring that the softmax effectively ignores those positions.
3. **Softmax**:
   - `attn_probs = torch.softmax(attn_scores, dim=-1)`: Converts the attention scores into probabilities using the softmax function. The probabilities indicate how much each key attends to each query.
4. **Attention Output**:
   - `output = torch.matmul(attn_probs, V)`: The final output is computed as the weighted sum of values (`V`), where the weights are the attention probabilities.

```python
def scaled_dot_product_attention(self, Q, K, V, mask=None):
    # Calculate attention scores
    attn_scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)
    # Apply mask if provided (useful for preventing attention to certain par
    if mask is not None:
        attn_scores = attn_scores.masked_fill(mask == 0, -1e9)
    # Softmax is applied to obtain attention probabilities
    attn_probs = torch.softmax(attn_scores, dim=-1)
    # Multiply by values to obtain the final output
    output = torch.matmul(attn_probs, V)
    return output
```

Multi-Head SA

`split_heads(self, x)`:

This method prepares the input for multi-head attention by splitting the embedding dimension into multiple heads.

1. **Input Shape**:
   - `x.size()` gives the shape as `(batch_size, seq_length, d_model)`.
2. **Reshaping**:
   - `x.view(batch_size, seq_length, self.num_heads, self.d_k)`: The input tensor is reshaped so that the dimension `d_model` is split into `num_heads` and `d_k` (the dimensionality per head).
3. **Transposing**:
   - `.transpose(1, 2)`: After splitting into heads, the tensor is transposed to bring the `num_heads` dimension before the `seq_length`, resulting in a shape of `(batch_size, num_heads, seq_length, d_k)`.

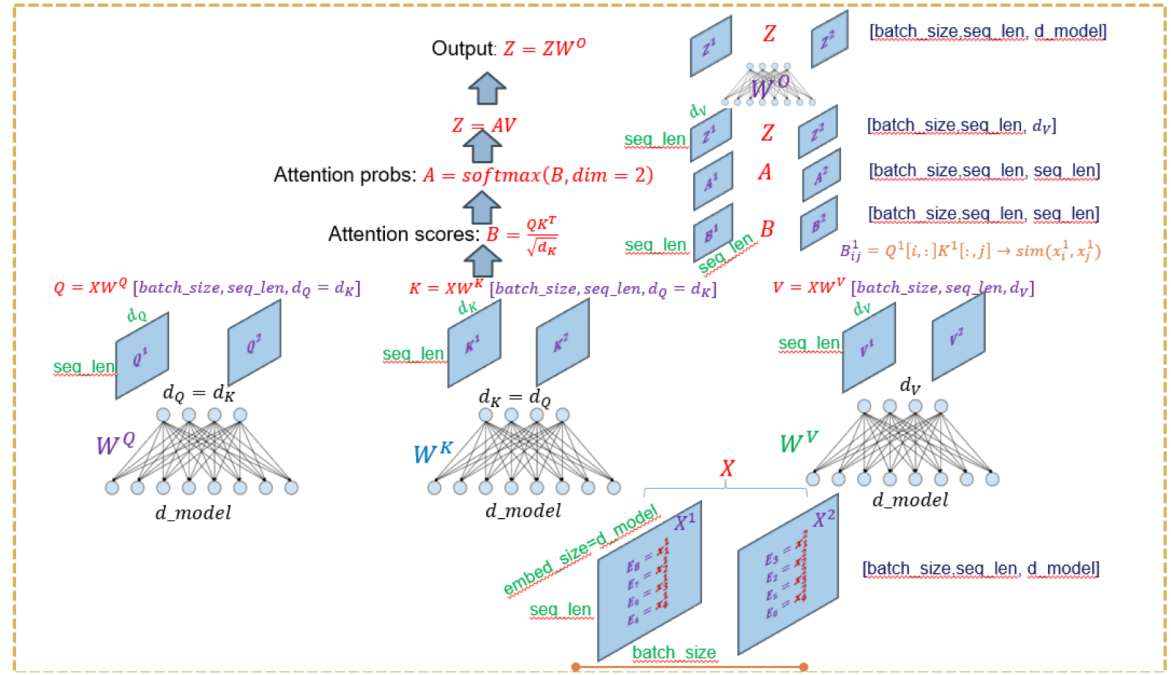
```python
def split_heads(self, x):
    # Reshape the input to have num_heads for multi-head attention
    batch_size, seq_length, d_model = x.size()
    return x.view(batch_size, seq_length, self.num_heads, self.d_k).transpose(1, 2)

def combine_heads(self, x):
    # Combine the multiple heads back to original shape
    batch_size, _, seq_length, d_k = x.size()
    return x.transpose(1, 2).contiguous().view(batch_size, seq_length, self.d_model)
```

`combine_heads(self, x)`:

This method combines the multiple attention heads back into the original embedding dimension.

1. **Input Shape**:
   - After multi-head attention, the tensor has a shape of `(batch_size, num_heads, seq_length, d_k)`.
2. **Reshaping**:
   - `x.view(batch_size, seq_length, self.d_model)`: The heads are merged back into the original shape, with `d_model = num_heads * d_k`.

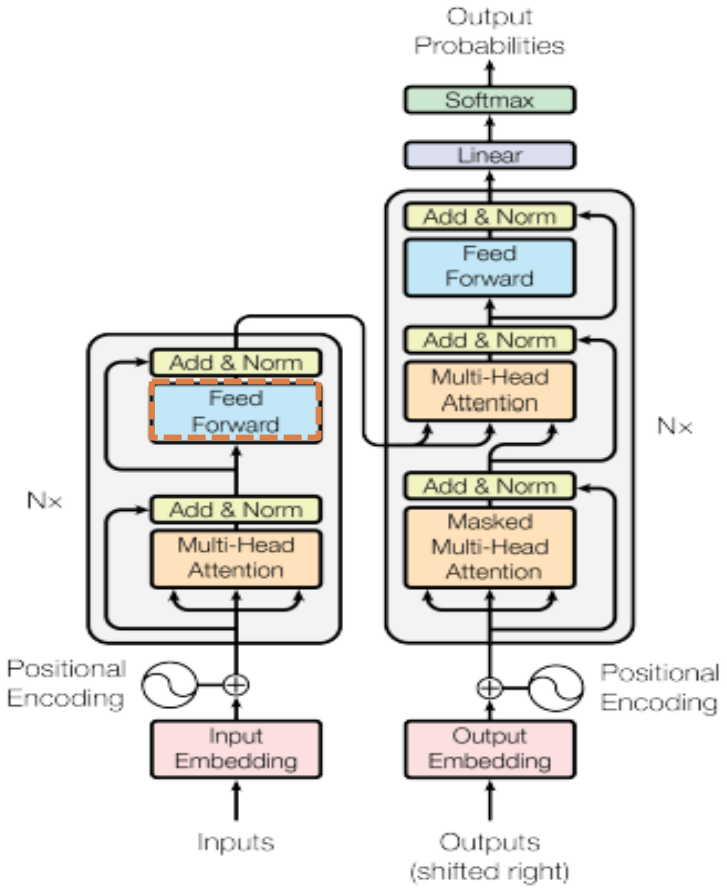



```python
def forward(self, Q, K, V, mask=None):
    # Apply linear transformations and split heads
    Q = self.split_heads(self.W_q(Q))
    K = self.split_heads(self.W_k(K))
    V = self.split_heads(self.W_v(V))

    # Perform scaled dot-product attention
    attn_output = self.scaled_dot_product_attention(Q, K, V, mask)

    # Combine heads and apply output transformation
    output = self.W_o(self.combine_heads(attn_output))
    return output
```

Transformer Encoder




```python
class EncoderLayer(nn.Module):
    def __init__(self, d_model, num_heads, d_ff, dropout):
        super(EncoderLayer, self).__init__()
        self.self_attn = MultiHeadAttention(d_model, num_heads)
        self.feed_forward = PositionWiseFeedForward(d_model, d_ff)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask):
        attn_output = self.self_attn(x, x, x, mask)
        x = self.norm1(x + self.dropout(attn_output))
        ff_output = self.feed_forward(x)
        x = self.norm2(x + self.dropout(ff_output))
        return x
```

```python
class PositionWiseFeedForward(nn.Module):
    def __init__(self, d_model, d_ff):
        super(PositionWiseFeedForward, self).__init__()
        self.fc1 = nn.Linear(d_model, d_ff)
        self.fc2 = nn.Linear(d_ff, d_model)
        self.relu = nn.ReLU()

    def forward(self, x):
        return self.fc2(self.relu(self.fc1(x)))
```

```python
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_seq_length):
        super(PositionalEncoding, self).__init__()

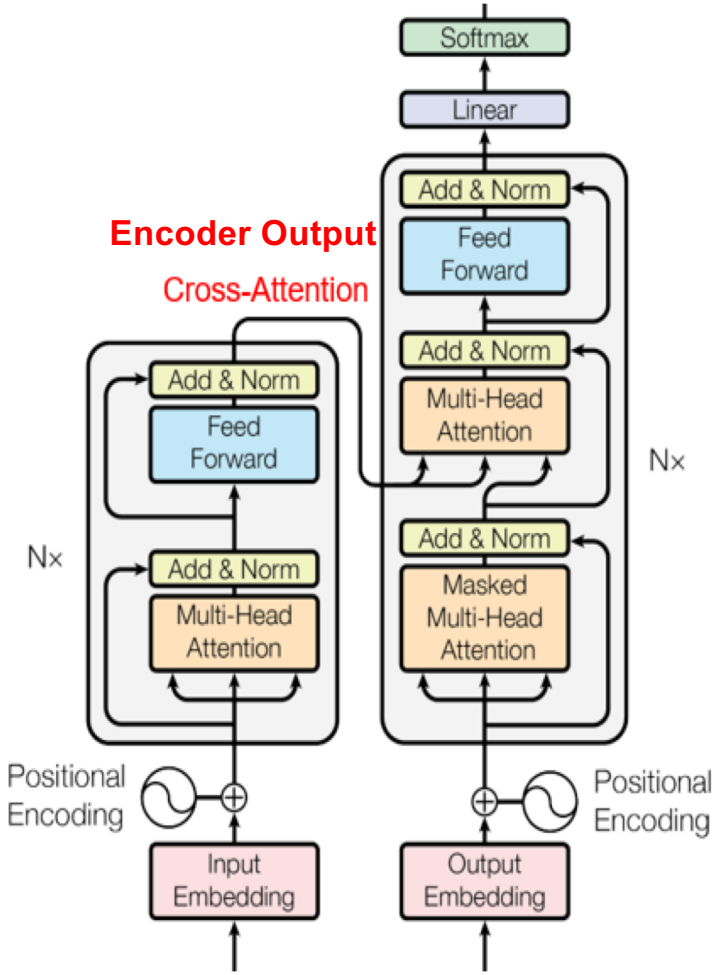
        pe = torch.zeros(max_seq_length, d_model)
        position = torch.arange(0, max_seq_length, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * -(math.log(10000.0) / d_model))

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

        self.register_buffer('pe', pe.unsqueeze(0))

    def forward(self, x):
        return x + self.pe[:, :x.size(1)]
```

Transformer Decoder

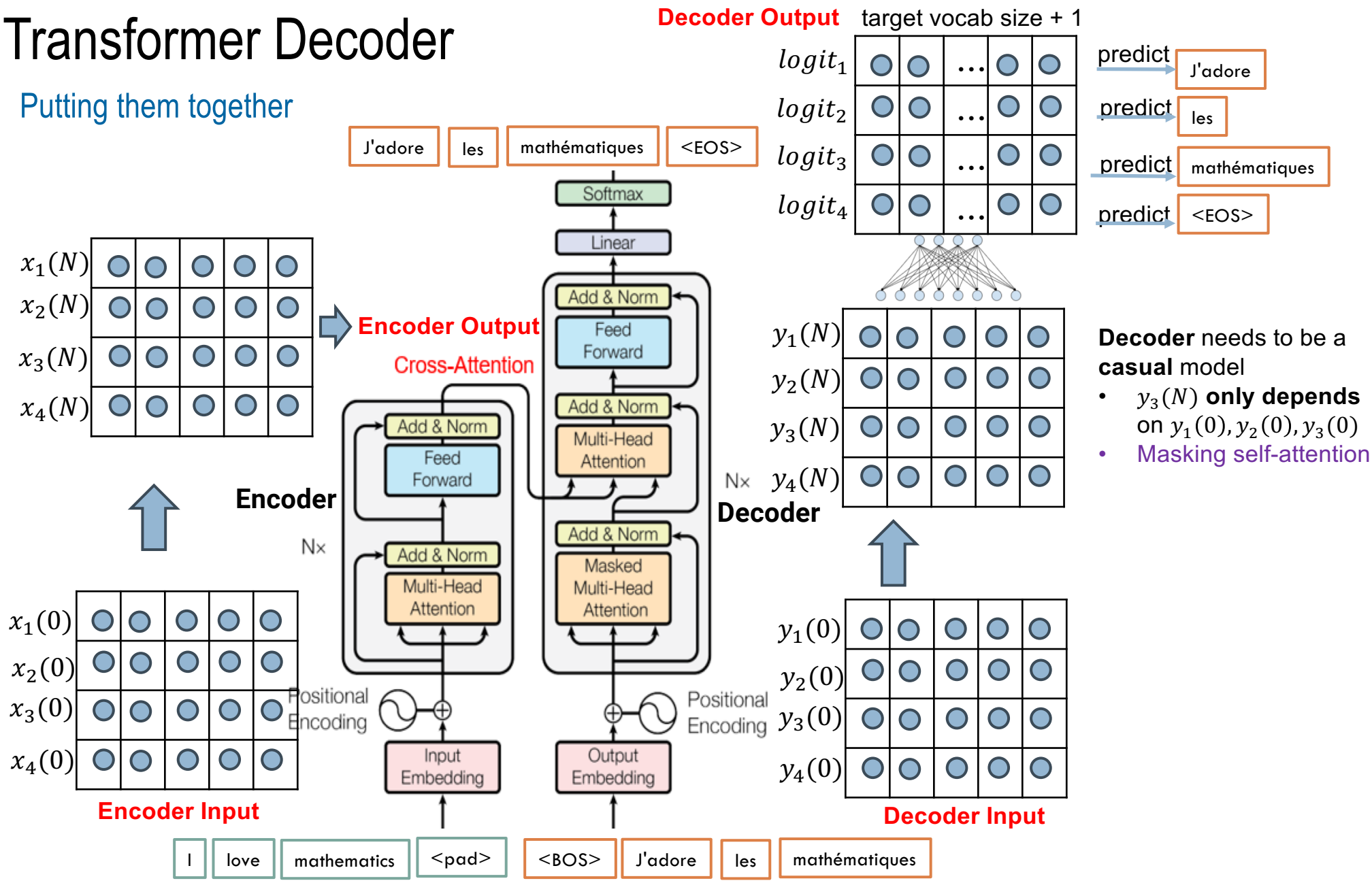



```python
class DecoderLayer(nn.Module):
    def __init__(self, d_model, num_heads, d_ff, dropout):
        super(DecoderLayer, self).__init__()
        self.self_attn = MultiHeadAttention(d_model, num_heads)
        self.cross_attn = MultiHeadAttention(d_model, num_heads)
        self.feed_forward = PositionWiseFeedForward(d_model, d_ff)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.norm3 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, enc_output, src_mask, tgt_mask):
        attn_output = self.self_attn(x, x, x, tgt_mask)
        x = self.norm1(x + self.dropout(attn_output))
        attn_output = self.cross_attn(x, enc_output, enc_output, src_mask)
        x = self.norm2(x + self.dropout(attn_output))
        ff_output = self.feed_forward(x)
        x = self.norm3(x + self.dropout(ff_output))
        return x
```

Transformer Decoder

Putting them together



Decoder needs to be a **casual** model

- $y_3(N)$ **only depends** on $y_1(0), y_2(0), y_3(0)$
- Masking self-attention

```python
class Transformer(nn.Module):
    def __init__(self, src_vocab_size, tgt_vocab_size, d_model, num_heads, num_layers,
                 d_ff, src_seq_length, tgt_seq_length, dropout):
        super(Transformer, self).__init__()
        self.encoder_embedding = nn.Embedding(src_vocab_size, d_model)
        self.decoder_embedding = nn.Embedding(tgt_vocab_size, d_model)
        self.src_positional_encoding = PositionalEncoding(d_model, src_seq_length)
        self.tgt_positional_encoding = PositionalEncoding(d_model, tgt_seq_length)


        self.encoder_layers = nn.ModuleList([EncoderLayer(d_model, num_heads, d_ff, dropout) for _ in range(num_layers)])
        self.decoder_layers = nn.ModuleList([DecoderLayer(d_model, num_heads, d_ff, dropout) for _ in range(num_layers)])

        self.fc = nn.Linear(d_model, tgt_vocab_size)
        self.dropout = nn.Dropout(dropout)

    def generate_mask(self, src, tgt):
        src_mask = (src != 0).unsqueeze(1).unsqueeze(2).to(device)
        tgt_mask = (tgt != 0).unsqueeze(1).unsqueeze(3).to(device)
        seq_length = tgt.size(1)
        # Create nopeak_mask on the same device as tgt
        nopeak_mask = (1 - torch.triu(torch.ones(1, seq_length, seq_length, device=device), diagonal=1)).bool()
        tgt_mask = tgt_mask & nopeak_mask
        return src_mask, tgt_mask

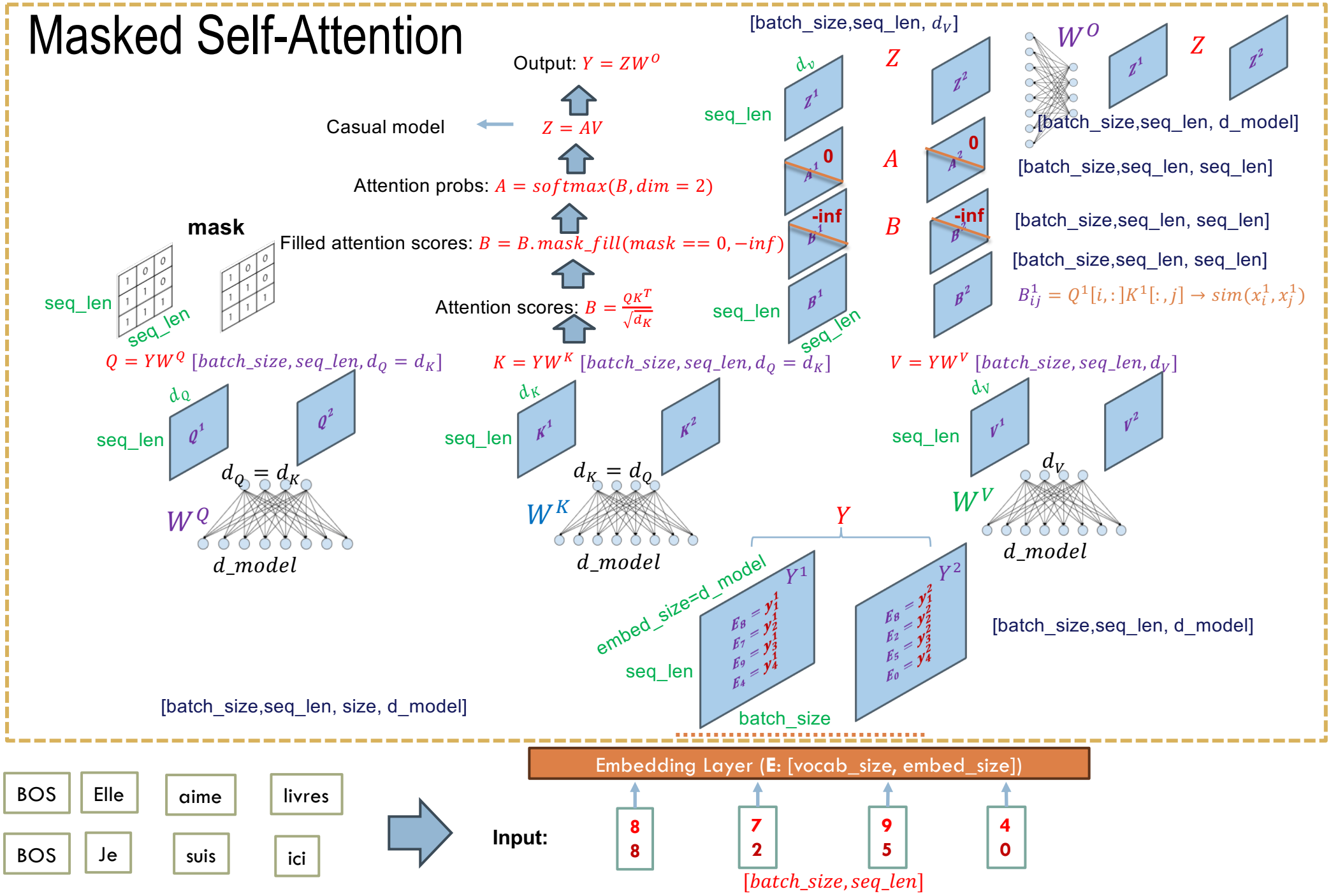
    def forward(self, src, tgt):
        src_mask, tgt_mask = self.generate_mask(src, tgt)
        src_embedded = self.dropout(self.src_positional_encoding(self.encoder_embedding(src)))
        tgt_embedded = self.dropout(self.tgt_positional_encoding(self.decoder_embedding(tgt)))

        enc_output = src_embedded
        for enc_layer in self.encoder_layers:
            enc_output = enc_layer(enc_output, src_mask)

        dec_output = tgt_embedded
        for dec_layer in self.decoder_layers:
            dec_output = dec_layer(dec_output, enc_output, src_mask, tgt_mask)

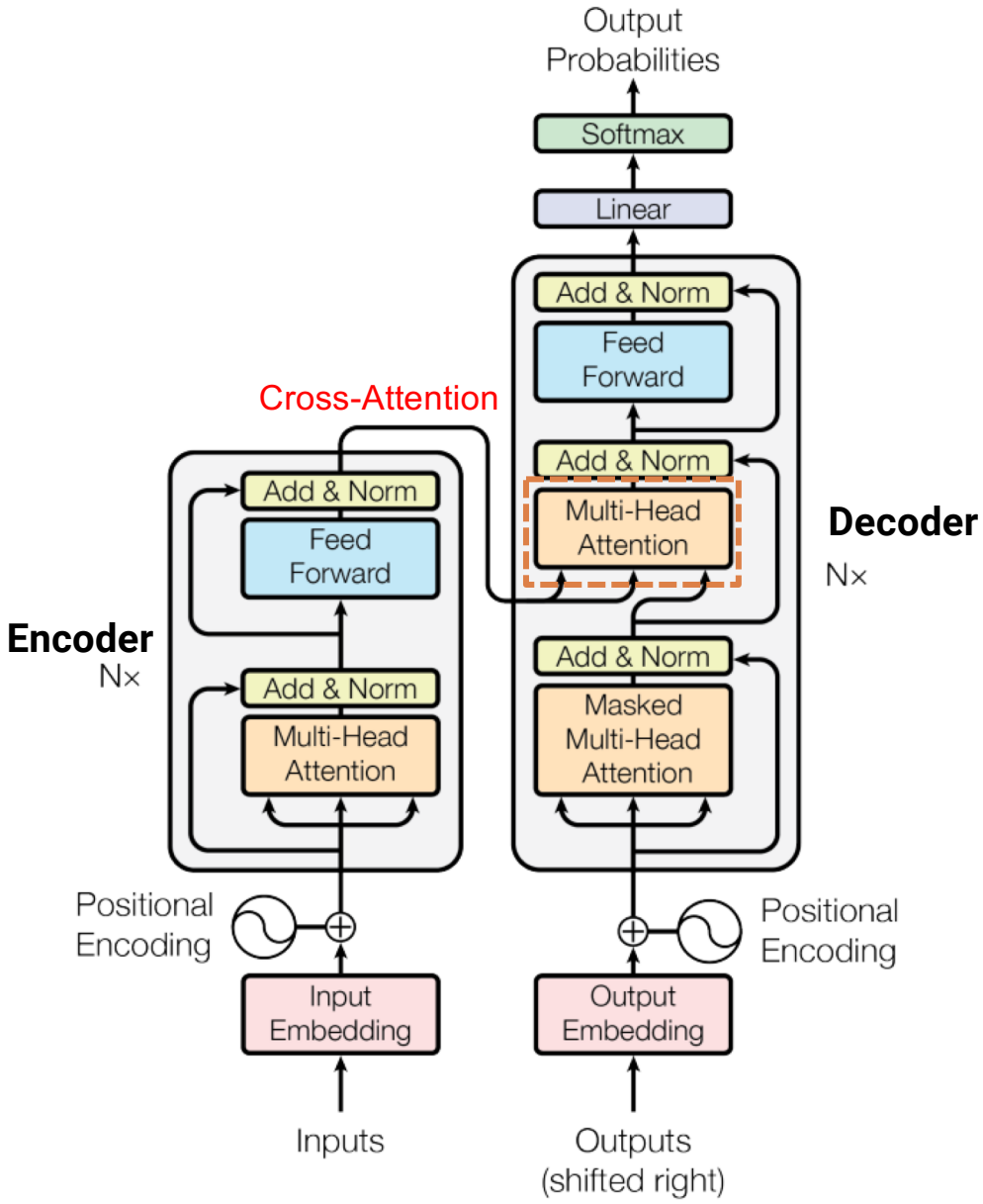
        output = self.fc(dec_output)
        return output
```

Masked Self-Attention

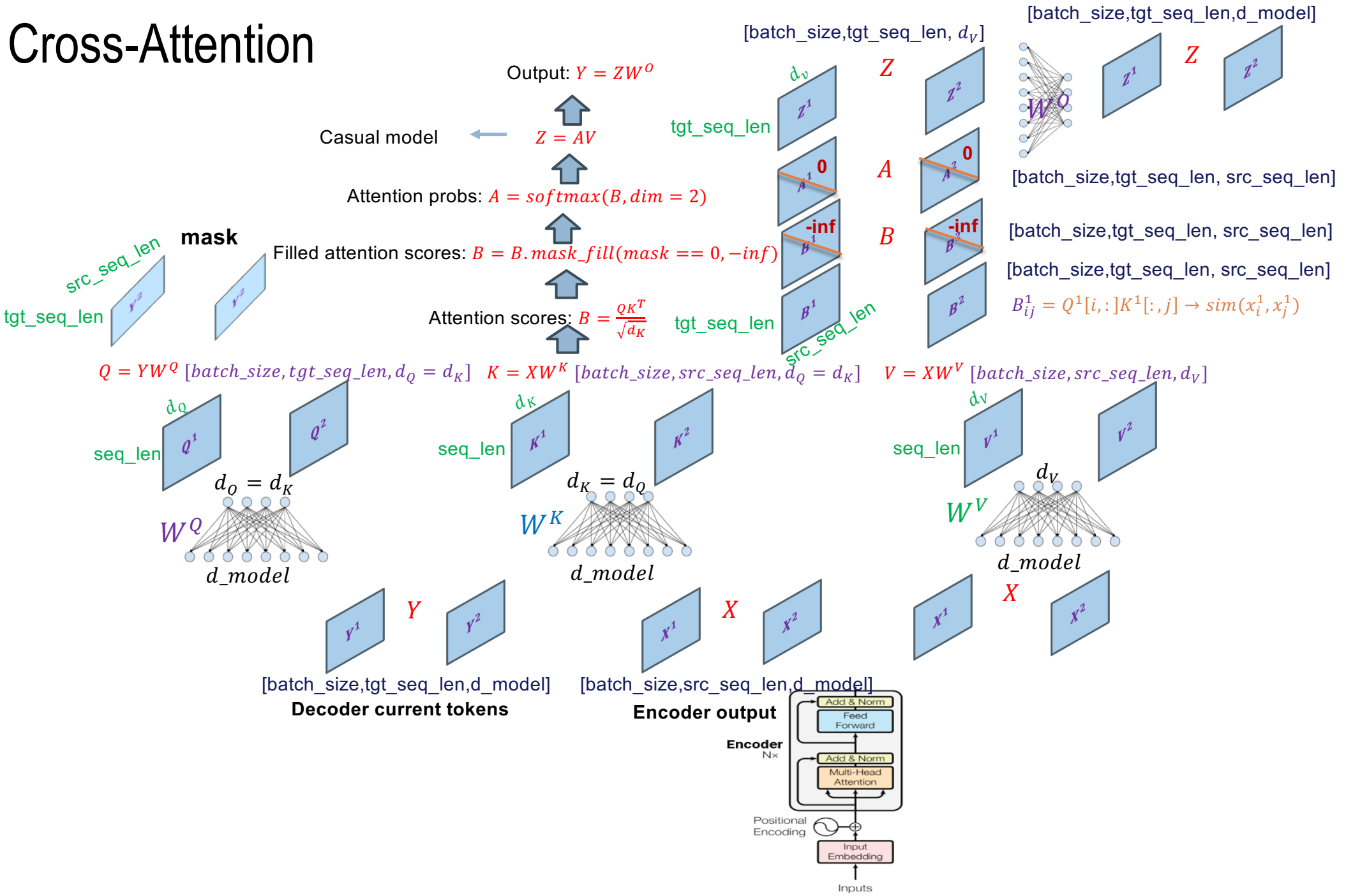


Transformer Decoder

Putting them together



Cross-Attention



Thanks for your attention!